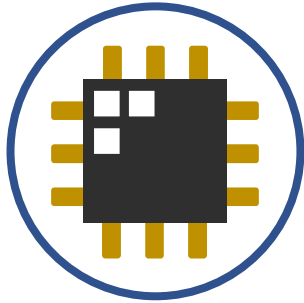


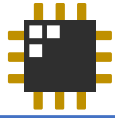


EEE 143: Switching Theory and Digital Logic Design

Lecture 6 – Combinational Logic Blocks Data Selectors, Encoders and Decoders

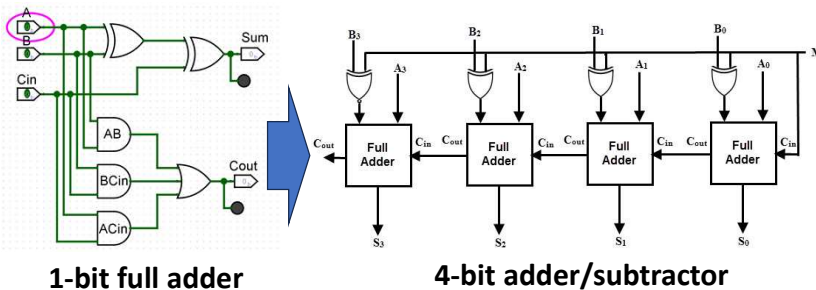
1

Recall that in our last meeting, we covered arithmetic, driven by the motivation to move up the abstraction level. Instead of focusing on logic gates and truth tables, we shifted our perspective to the desired functionality specifically, addition, subtraction, and comparison. The key question was: what logic blocks do we need to implement these functions?



Recap: Moving towards higher abstractions

- We're moving towards larger and more complex digital blocks, focusing more on the **functionality** rather than gate-level design
 - We started with binary arithmetic



1-bit full adder

4-bit adder/subtractor

Let's cover the other combinational logic blocks!

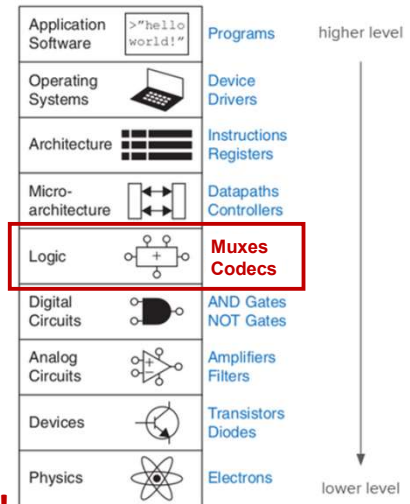
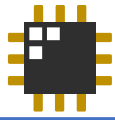


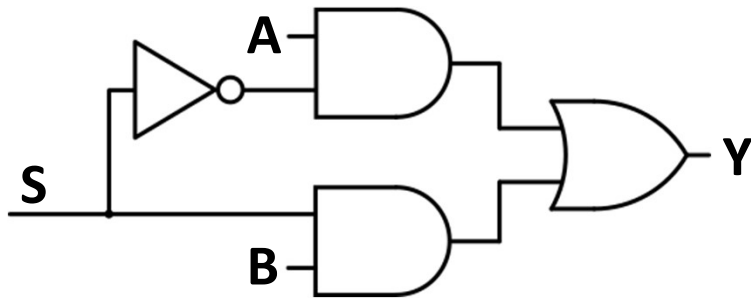
Image credit: Digital Design and Computer Architecture (2nd ed.) by Harris, H., Harris, S. (2013). 2

Now, we'll continue talking about logic blocks, focusing on data selectors, multiplexers, demultiplexers, decoders, and encoders. We'll start by understanding what these components do at a high level. Then, we'll work our way back to see how they can be represented using logic expressions and how we can build them as logic blocks.



Remember this?

- We can make combinations of logic gates to make more complex logic!

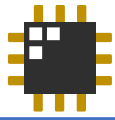


$$Y = S'A + SB$$

Inputs			Out
S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

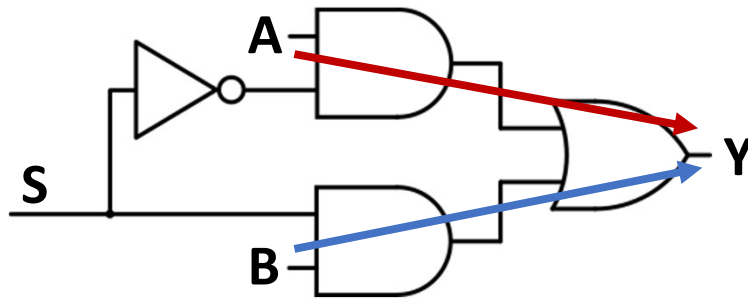
3

This was one of the complex logic that we analyzed back in Lecture 2. We started from the circuit and then derived the Boolean expression corresponding to that circuit. Then, we derived the truth table to better explain the functionality.



Multiplexers

- Multiplexer (mux) is a circuit that selects data from *one of many* (2^n) inputs and directs it to a *single* output using an n-bit selector



If $S = 0$, $Y = A$

If $S = 1$, $Y = B$

$$Y = S'A + SB$$

Inputs			Out
S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

4

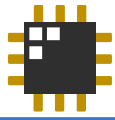
If you can remember how we explained the functionality of the circuit, we identified that the output basically just copies the value of A whenever S is 0. If S is 1 on the other hand, the output copies the value of B. You can observe this from the truth table on the right. Highlighted in red are the rows corresponding to $S = 0$, and highlighted in blue are the rows corresponding to $S = 1$. Taking a close look at the behavior of Y relative to the inputs, we see that Y just copies whatever A is in the red region, and Y copies whatever B is in the blue region. This reiterates the high-level behavior that we observed.

High-level description:

If $S = 0$, select A (Y copies the value of A)

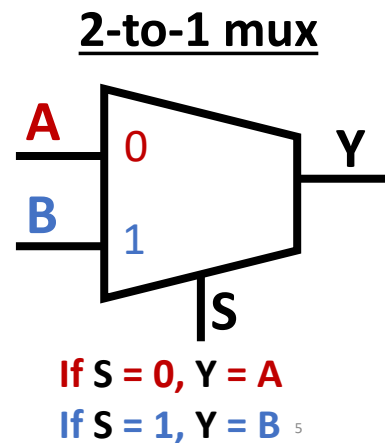
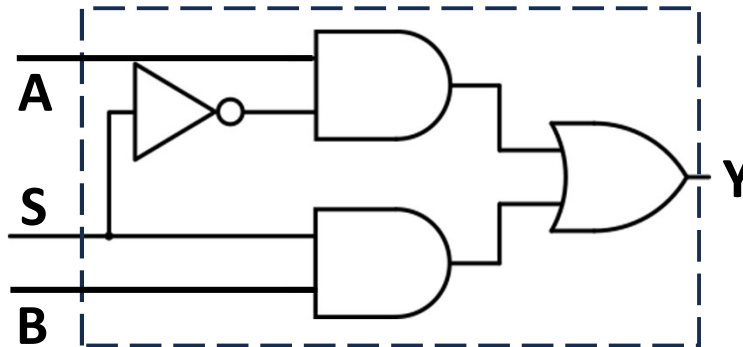
If $S = 1$, select B (Y copies the value of B)

This logic block is called a multiplexer, which selects from one of many (2^n) inputs and directs it to a single output using an n-bit selector.

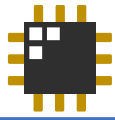


Multiplexers

- Multiplexer (mux) is a circuit that selects data from *one of many* (2^n) inputs and directs it to a *single* output using an n-bit selector



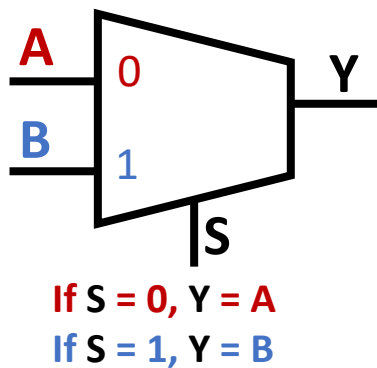
Now, since we have identified the unique functionality of this circuit, we can lump all of those logic gates together as a single box. When considering logic blocks, we tend to increase the level of abstraction and not show the component blocks anymore. At this point, given that we know what the functionality of the block is, we don't have to draw the corresponding logic gate equivalent circuit anymore. This simplifies things as we implement more complex logic.



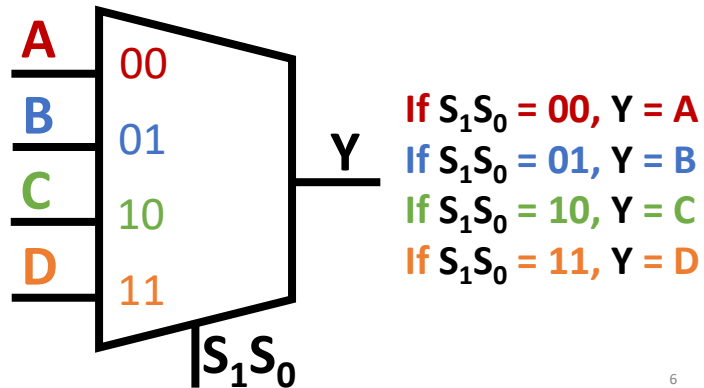
Multiplexers

- Multiplexer (mux) is a circuit that selects data from *one of many* (2^n) inputs and directs it to a *single* output using an n-bit selector

2-to-1 mux



4-to-1 mux



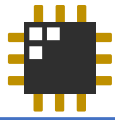
6

The functionality of a multiplexer is not limited to only two inputs. We can extend the concept by increasing the number of selector bits (the bottom lines) as shown in the diagram. Since we identified earlier that the S signal basically selects which of the two inputs will pass through the output, we can extend that concept by using two bits for the S signal, now S_1S_0 , which makes the multiplexer able to select between four different inputs, corresponding to the different S_1S_0 combinations. This now becomes a 4-to-1 multiplexer, as it selects 4 different inputs to map to a single output.

1-bit selector, 2^1 inputs

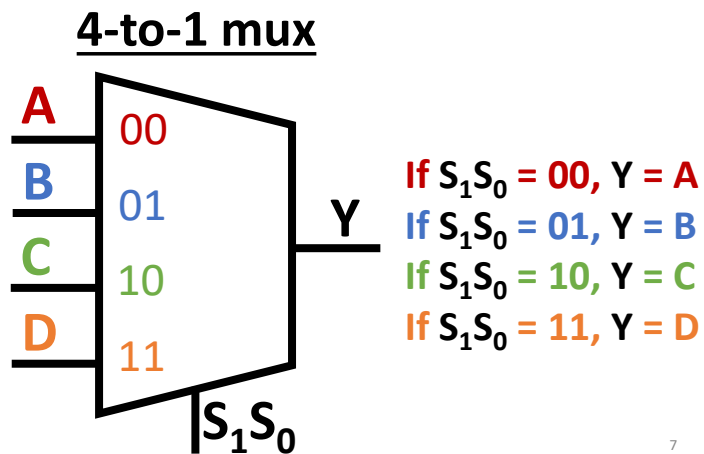
2-bit selector, 2^2 inputs

The inputs being selected corresponding to the different S_1S_0 combinations are shown with their color correspondence for easy identification for now.



Multiplexers

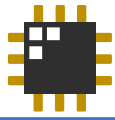
- Write the equivalent Boolean expression for Y.



So, just a practice. While I said earlier that once we move on to logic blocks we tend not to draw the logic gate equivalent circuit anymore. However, it is always nice to still know what is happening in the backend as we explore more complex blocks. So now, given the functionality of the 4-to-1 multiplexer, which is shown on the right, what is the equivalent Boolean expression for the output.

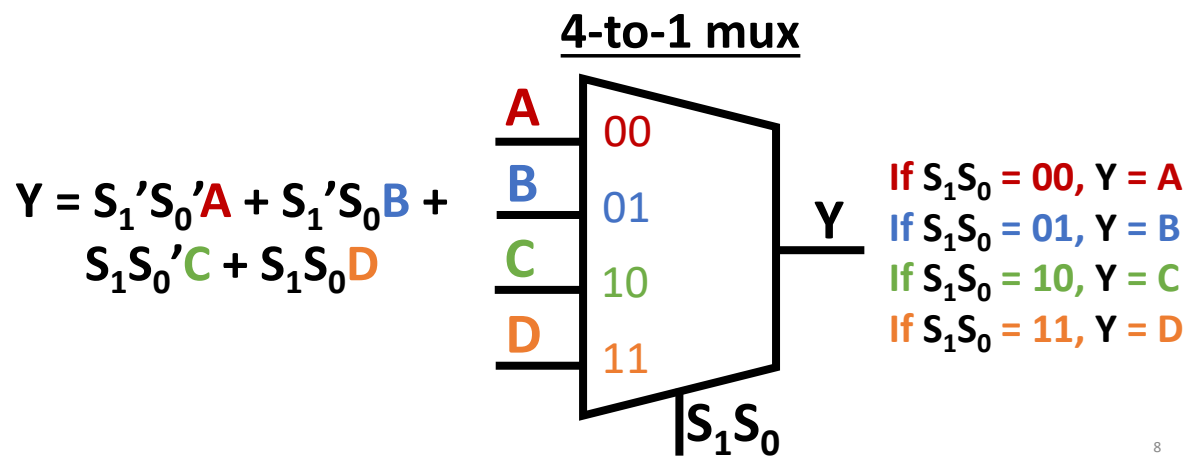
Note how different S_1S_0 combinations map one of the inputs to the output. For example, if S_1S_0 is 00, then Y just copies whatever A is. This implies the Boolean expression $S_1'S_0'A$. Do this for all other combinations of S_1S_0 and their corresponding inputs and we will end up with.

$$Y = S_1'S_0'A + S_1'S_0B + S_1S_0'C + S_1S_0D$$

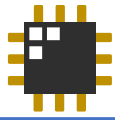


Multiplexers

- Write the equivalent Boolean expression for Y.

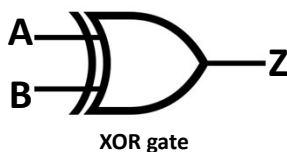


This is the Boolean expression for the output of a 4-to-1 multiplexer.



Truth tables using multiplexers

- Multiplexers can be used to implement truth tables!

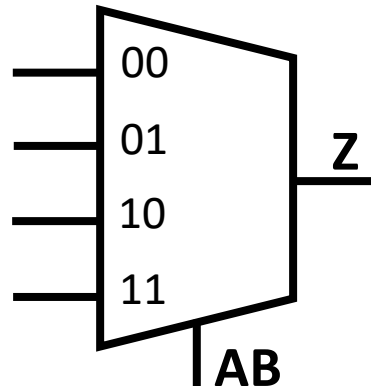


XOR gate

$$Z = A \oplus B$$

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

What should the multiplexer inputs be so that this block follows the XOR logic truth table?



9

Again, a multiplexer is a circuit that selects one of several inputs based on an n-bit selector. Since a truth table defines how outputs respond to different input combinations, we can use a multiplexer to implement any truth table by properly assigning its inputs.

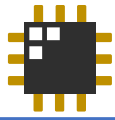
Now, let's examine the truth table shown on the left, which represents the XOR logic. The key question is: What should the multiplexer inputs be to make it follow the XOR truth table?

For this, we use a multiplexer with a two-bit selector (A and B) and four inputs. As mentioned earlier, the selector values correspond to the possible input combinations in the truth table: 00, 01, 10, and 11. Each of the four multiplexer inputs determines the output for its respective selector combination.

Since we are implementing XOR logic, the correct output values for these selector inputs are 0, 1, 1, and 0, respectively. By assigning these values to the multiplexer inputs, we ensure that it behaves like an XOR gate.

Hence,

00 -> 0
01 -> 1
10 -> 1
11 -> 0



Truth tables using multiplexers

- Multiplexers can be used to implement truth tables!

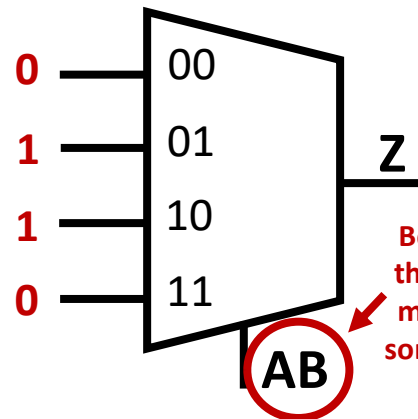


XOR gate

$$Z = A \oplus B$$

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

XOR logic using 4-to-1 multiplexer



Be careful about the order here, it might matter for some truth tables!

10

Again, the inputs for the corresponding AB combinations are as follows:

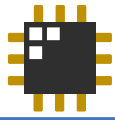
00 -> 0

01 -> 1

10 -> 1

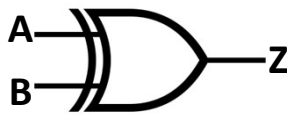
11 -> 0

Take Note: Be careful with the order of selector combinations! If the inputs are assigned incorrectly, the multiplexer will not produce the expected XOR output. Always double-check that the selector values match the correct positions in the truth table to avoid mistakes.



Truth tables using multiplexers

- Multiplexers can be used to implement truth tables!

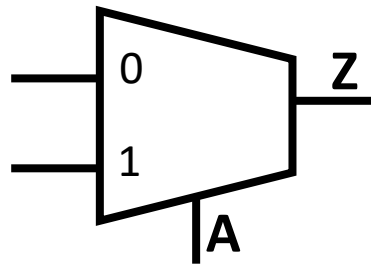


XOR gate

$$Z = A \oplus B$$

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

What should the multiplexer inputs be so that this block follows the XOR logic truth table?



11

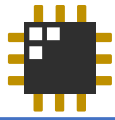
Previously, we used a **4-to-1 multiplexer** to implement XOR logic. But what if we try using a **2-to-1 multiplexer** instead? This is actually possible by treating one of the variables as the **select signal**, in this case, **A**, and determining the appropriate multiplexer inputs based on the desired output.

Let's revisit the truth table.

- When **A = 0**, the output is **1** if **B = 1**, and **0** if **B = 0**. This means the output simply follows **B**.
- When **A = 1**, the output is **1** if **B = 0**, and **0** if **B = 1**. Here, the output is the **complement of B (B')**.

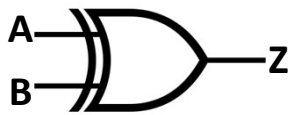
From this, we can conclude:

- If **A = 0**, then **Z = B**.
- If **A = 1**, then **Z = B'**.



Truth tables using multiplexers

- Multiplexers can be used to implement truth tables!

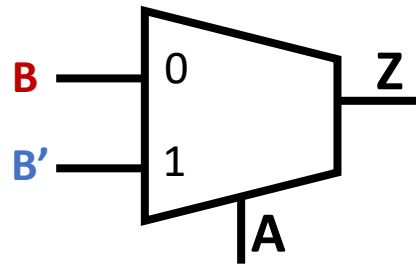


XOR gate

$$Z = A \oplus B$$

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

XOR logic using 2-to-1 multiplexer



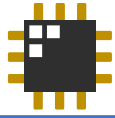
Try doing it using B as the selector instead!

12

Again, the corresponding input for each selector signal is:

- If $A = 0$, then $Z = B$.
- If $A = 1$, then $Z = B'$.

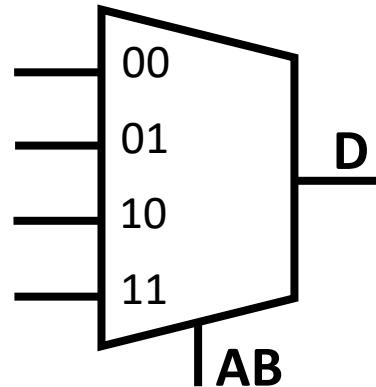
By assigning these values to the multiplexer inputs, we effectively implement XOR using a **2-to-1 multiplexer**!



Truth tables using multiplexers

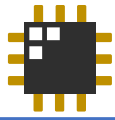
- Implement the following truth table using a **4-to-1 multiplexer**

Inputs			Out
A	B	C	D
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0



13

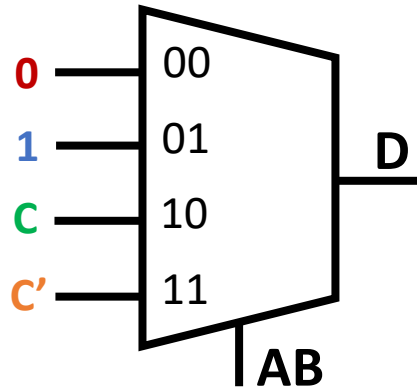
As a practice, let's try implementing this 3-variable truth table using a 4-to-1 mux. Given that we have a 4-to-1 mux, it means that we have 2 control lines. As such, one of the function inputs will be included in the multiplexer input. For this case, let us just choose C as the input to be connected to the mux input since A and B are directly connected to the selector lines.



Truth tables using multiplexers

- Implement the following truth table using a **4-to-1 multiplexer**

Inputs				Out
A	B	C	D	
0	0	0	0	0
0	0	1	0	
0	1	0	1	1
0	1	1	1	
1	0	0	0	C
1	0	1	1	
1	1	0	1	C'
1	1	1	0	

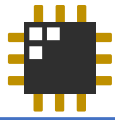


Try doing it using another pair of variables as the selector instead!

14

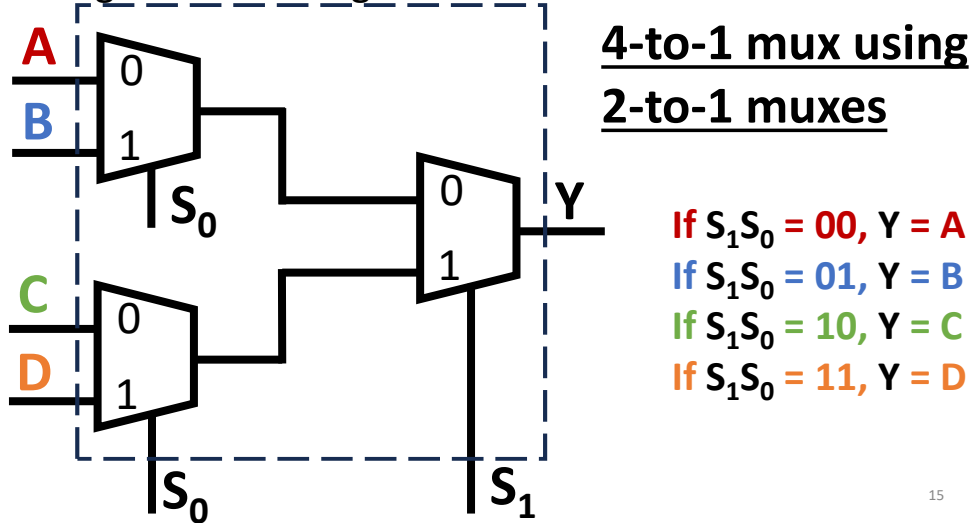
To do this, we have drawn a line separating C from the other inputs. We use what is called the variable entered mapping (VEM) technique to merge C to the output. Take for example the case when $A=0$ and $B=0$: regardless of the value of C, D is always 0. thus, for $A=B=0$, the effective (merged C and D) output is 0. For the case where $A=0$ and $B=1$, output is 1 (regardless of the value of C). Let us then take the case when $A=1$ and $B=0$. when C is 0 (C'), D is 0 and when C is 1 (C), D is 1. We can re-write this as $C'(0) + C(1) = C$. Lastly, we consider the case when $A=B=1$. The expression for the output will be $C'(1) + C(0) = C'$.

Then we can use the same method we used previously and connect inputs A and B to the selector lines (note again that the order is important and therefore you have to label your multiplexers properly) and then connect the 'effective' (merged) output to the multiplexer inputs as shown on the right.



Cascading multiplexers

- You can make larger muxes using smaller muxes!



15

The cool thing about multiplexers is that we can create larger multiplexers using smaller ones. In this diagram, we have a 4-to-1 multiplexer being implemented using 2-to-1 multiplexers.

We can verify the functionality by looking at how the subblocks behave given the inputs. For instance, if S_0 and S_1 are both 0, then the multiplexers on the left would be selecting the A input and the C input at the same time, while the multiplexer on the right would be selecting the one corresponding to the 0 input, which was A as we identified earlier. Thus, we can say that for $S_1S_0 = 00$, then Y just copies A, which is the correct behavior of the 4-to-1 multiplexer.

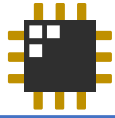
We can extend this description for all the other cases.

For $S_1S_0 = 01$: The multiplexers on the right would be selecting the B input and the D input at the same time, while the multiplexer on the left would be selecting the one corresponding to the 0 input, which was B as we identified earlier. Thus, we can say that for $S_1S_0 = 01$, then Y just copies B, which is the correct behavior of the 4-to-1 multiplexer.

For $S_1S_0 = 10$: The multiplexers on the right would be selecting the A input and the C input at the same time, while the multiplexer on the left would be selecting the one corresponding to the 1 input, which was C as we identified earlier. Thus, we can say that for $S_1S_0 = 10$, then Y just copies C, which is the

correct behavior of the 4-to-1 multiplexer.

For **S1S0 = 11**: The multiplexers on the right would be selecting the B input and the D input at the same time, while the multiplexer on the left would be selecting the one corresponding to the 1 input, which was D as we identified earlier. Thus, we can say that for S1S0 = 11, then Y just copies D, which is the correct behavior of the 4-to-1 multiplexer.



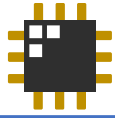
Truth tables using cascading multiplexers

- Implement $F(A,B,C,D) = \sum m(0, 3, 5, 6, 8, 9, 14, 15)$ using **4-to-1 muxes**

16

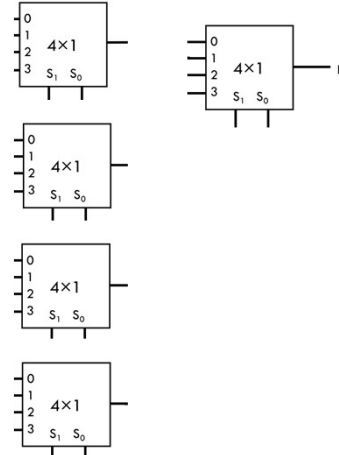
As we all know, the truth table defines how a logic function corresponds to different input combinations. One way to implement these functions is by using cascading multiplexers, which can simplify complex logic expressions while reducing the need for multiple gates.

Before we start with the example, let's recall the **Sum of Products (SOP) form**. The SOP representation expresses a Boolean function as a sum (OR) of minterms, where each minterm (product term) corresponds to a row in the truth table where the output is **1**.



Truth tables using cascading multiplexers

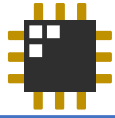
- Implement $F(A,B,C,D) = \sum m(0, 3, 5, 6, 8, 9, 14, 15)$ using **4-to-1 muxes**



17

Again, a 4-to-1 multiplexer has two-bit selector and four inputs. We can simplify the circuit by choosing two variables as select signals and setting the inputs accordingly.

- We choose A and B as the select signals for the final MUX.
- The data inputs of this final MUX will come from two smaller 4-to-1 MUXes, which use C and D as their select signals.

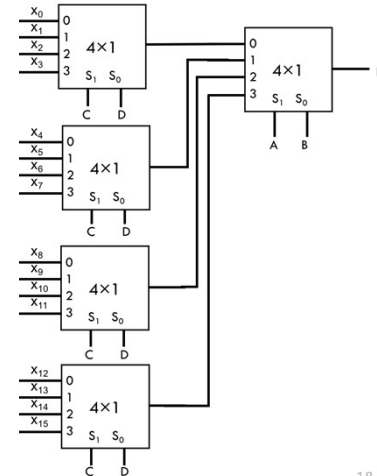


Truth tables using cascading multiplexers

- Implement $F(A,B,C,D) = \sum m(0, 3, 5, 6, 8, 9, 14, 15)$ using **4-to-1 muxes**

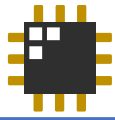
Inputs				Out
A	B	C	D	F
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	

Inputs				Out
A	B	C	D	F
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	



18

In this case, the most significant input bits serve as selector of the mux closest to the output. Notice that when $A=B=0$, F will get the value from input 0, which in turn is connected to the top-most mux. Thus, m_0 - m_3 will be the inputs for the topmost mux. Note that the selector for all muxes on the left are the same.

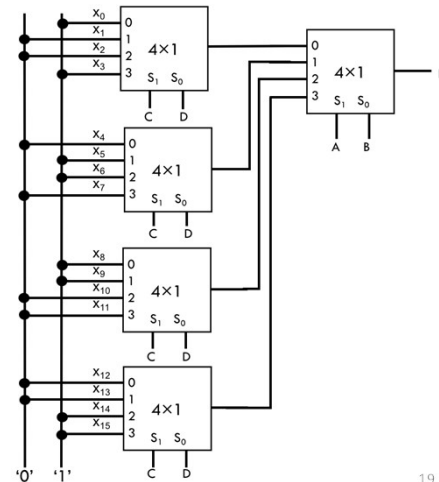


Truth tables using cascading multiplexers

- Implement $F(A,B,C,D) = \sum m(0, 3, 5, 6, 8, 9, 14, 15)$ using **4-to-1 muxes**

Inputs				Out
A	B	C	D	F
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0

Inputs				Out
A	B	C	D	F
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

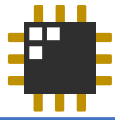


19

We now group the minterms based on the values of **A and B**:

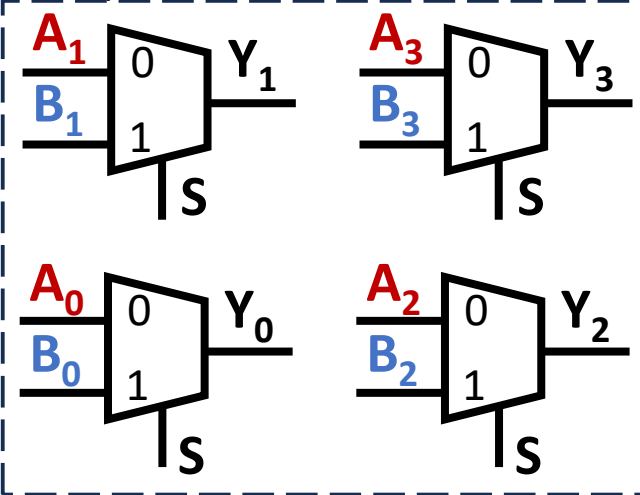
- When **A = 0, B = 0** → We look at minterms **0, 1, 2, 3** → The output is **1** for **m(0, 3)**
- When **A = 0, B = 1** → We look at minterms **4, 5, 6, 7** → The output is **1** for **m(5, 6)**
- When **A = 1, B = 0** → We look at minterms **8, 9, 10, 11** → The output is **1** for **m(8, 9)**
- When **A = 1, B = 1** → We look at minterms **12, 13, 14, 15** → The output is **1** for **m(14, 15)**

Each of these four groups is implemented using a **4-to-1 MUX** with **C and D** as select inputs. The outputs of these MUXes are then sent to the final **4-to-1 MUX**, where **A and B** act as the select inputs.



Multi-bit multiplexers

- Multiplexers can be used in parallel to handle several bits at once!



4-bit 2-to-1 mux

If $S = 0$, $Y_3Y_2Y_1Y_0 = A_3A_2A_1A_0$

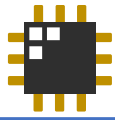
If $S = 1$, $Y_3Y_2Y_1Y_0 = B_3B_2B_1B_0$

All muxes here *run* in parallel

20

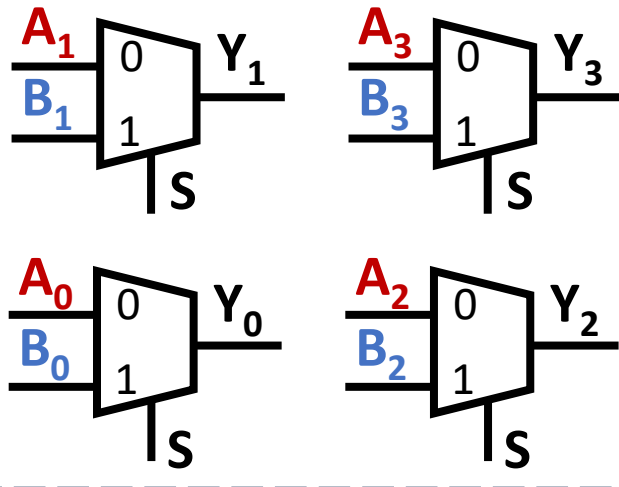
Multiplexers can also select several bits at once by having them in parallel. Note that the multiplexers we explored earlier only had 1-bit inputs for A,B (and C,D for 4-to-1 mux). Now, if we want to extend this, then we just create several instances of the same multiplexer and have each of them handle individual bits of our binary string.

For example, this diagram shows how a 1-bit 2-to-1 mux, that we already know, can be extended as a 4-bit 2-to-1 mux when there's four of them running in parallel, each multiplexer handling a separate bit. In this example, if S is 0, all the multiplexers will be selecting the inputs A_x to be passed to the output Y_x . Then, if S is 1, all the multiplexers will be selecting the inputs B_x to the output Y_x . This is how we implement a multi-bit multiplexer.



Multi-bit multiplexers

- Multiplexers can be used in parallel to handle several bits at once!

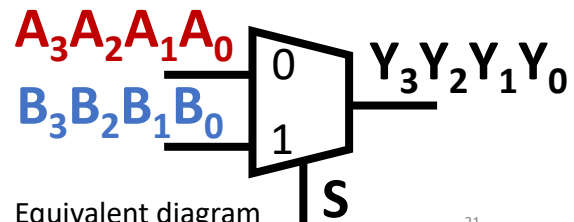


4-bit 2-to-1 mux

If $S = 0$, $Y_3Y_2Y_1Y_0 = A_3A_2A_1A_0$

If $S = 1$, $Y_3Y_2Y_1Y_0 = B_3B_2B_1B_0$

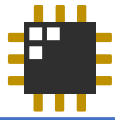
All muxes here *run* in parallel



Equivalent diagram

21

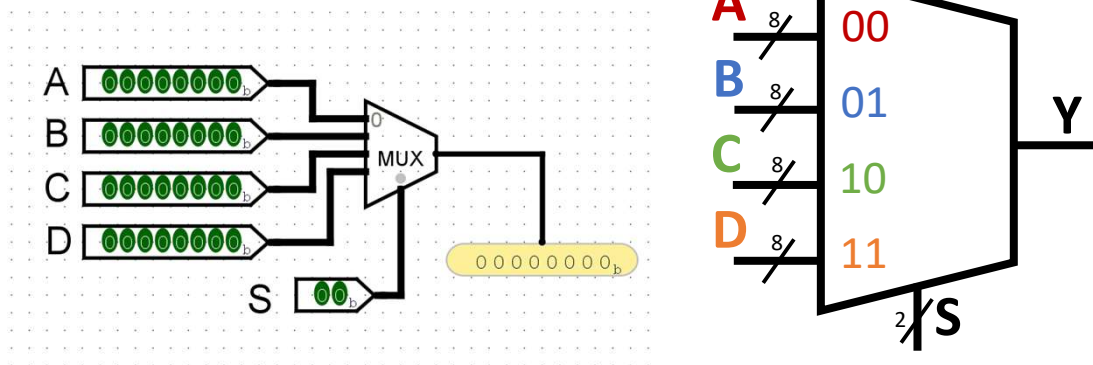
And here's the equivalent diagram for that one. We can lump all of those multiplexer subblocks and just draw a single multiplexer that indicates that it can handle several bits at the same time.



Multi-bit multiplexers

- Multiplexers can be used in parallel to handle several bits at once!

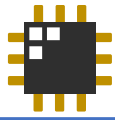
8-bit 4-to-1 mux



When considering multi-bit muxes, we sometimes don't put the number of bits to make the drawing simpler.

Here's a demonstration of the 8-bit 4-to-1 multiplexer. Note that for simplicity, we don't try to write all the bits anymore in the multiplexer diagram, since that would end up being too long, especially if you have a lot of bits to be considered in parallel. In this case, we assume that A, B, C, D, and Y are all 8-bits long, and therefore the multiplexer should be an 8-bit 4-to-1 multiplexer, without explicitly writing all the 8 bits. Moreover, since we can see that the multiplexer has 4 inputs, then we can say that S is 2-bits, without explicitly writing how many bits it is, since it has to select one of the inputs to be passed to the output.

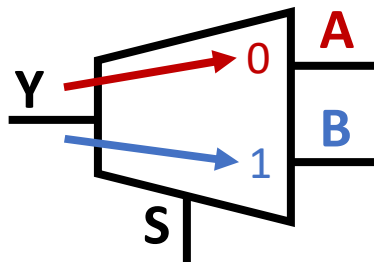
As you can see in the demo, different values of A, B, C, and D were selected depending on what the value of S is. The input corresponding to S=00 is still the input A (at the top) in this demo and the assignment just continues downward in increasing binary value. This is consistent with the multiplexer drawing on the right.



Demultiplexers

- We can technically do the reverse functionality!

Depending on the selector, direct Y to A or direct Y to B



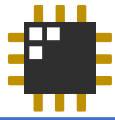
Inputs		Outputs	
S	Y	A	B
0	0	0	0
0	1	1	0
1	0	0	0
1	1	0	1

Note: Y is the input here (S is still an input), A and B are outputs

23

We can technically reverse the functionality of a multiplexer and have a block that does that. In this diagram, Y is now being passed to either A or B, depending on the selector S. Note that in this case, Y is now the input of the logic block while A and B are the outputs. S remains an input of the block as it has to select the mapping that needs to be performed. If S is 0, whatever the value of Y is will be passed to the output A. If S is 1, then it is mapped to the output B instead.

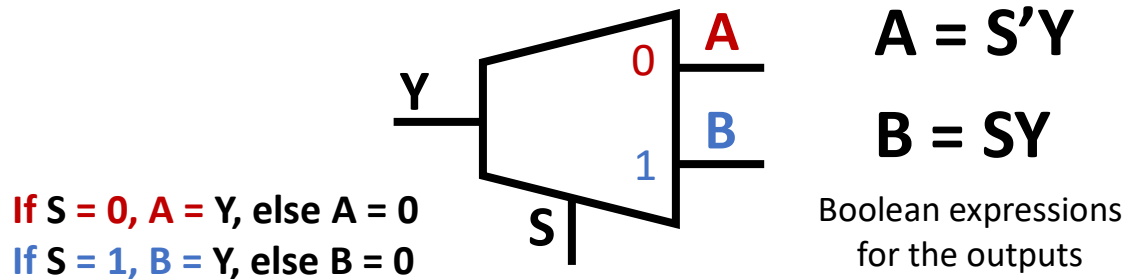
The truth table on the right describes the exact functionality. When S = 0, A copies exactly what Y is, while B remains 0. When S = 1, B copies exactly what Y is, while A remains 0.



Demultiplexers

- Demultiplexers direct the *single* input to *one of many* (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-2 demux

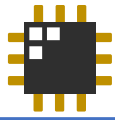


Note: Y is the input here (S is still an input), A and B are outputs

24

This logic block that we were describing is called a demultiplexer, where the functionality is somewhat the reverse of what a multiplexer is. It directs the single output to one of many outputs (2^n) corresponding to the n-bit selector.

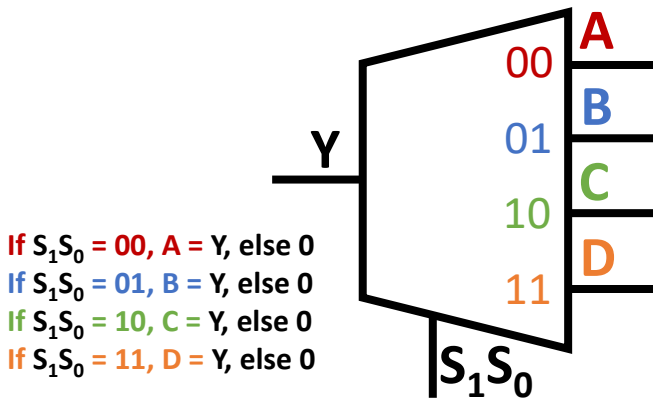
Here's the Boolean expression counterpart of the truth table shown in the previous slide. Since A copies whatever Y is when S is 0, then we have the expression $A = S'Y$. Since B copies whatever Y is when S is 1, then we have the expression $B = SY$. If the conditions are not satisfied, B remains 0 whenever S is 0 and A remains 0 whenever S is 1, which exactly corresponds to the high-level description of the logic block we had.



Demultiplexers

- Demultiplexers direct the *single* input to *one of many* (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



Inputs			Outputs			
S_1	S_0	Y	A	B	C	D
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

25

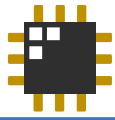
As a practice, we should be able to write the truth table of a 1-to-4 demultiplexer given the high-level functionality. The truth table is already colored for convenience, highlighting the inputs that matter given a specific condition (value of S_1 and S_0).

As long as $S_1S_0 = 00$, A copies Y, while the other outputs remain 0.

If $S_1S_0 = 01$, B copies Y, while other outputs remain 0.

If $S_1S_0 = 10$, C copies Y, while other outputs remain 0.

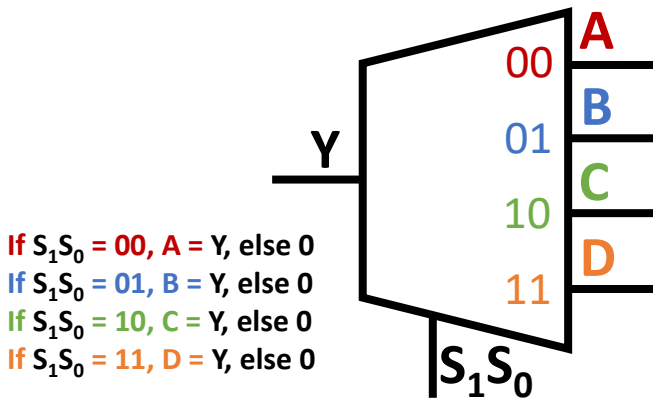
If $S_1S_0 = 11$, D copies Y, while other outputs remain 0.



Demultiplexers

- Demultiplexers direct the *single* input to *one of many* (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



Inputs			Outputs			
S_1	S_0	Y	A	B	C	D
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	0	0
0	1	1	0	1	0	0
1	0	0	0	0	0	0
1	0	1	0	0	1	0
1	1	0	0	0	0	0
1	1	1	0	0	0	1

26

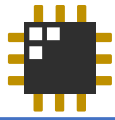
Here's the completed truth table given the high-level functionality description of a 1-to-4 demultiplexer.

As long as $S_1S_0 = 00$, A copies Y, while the other outputs remain 0.

If $S_1S_0 = 01$, B copies Y, while other outputs remain 0.

If $S_1S_0 = 10$, C copies Y, while other outputs remain 0.

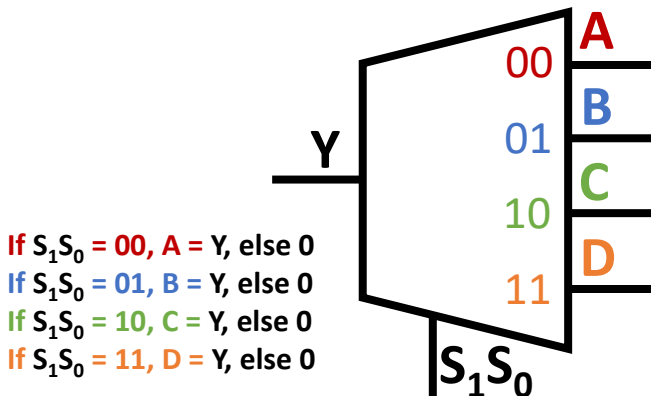
If $S_1S_0 = 11$, D copies Y, while other outputs remain 0.



Demultiplexers

- Demultiplexers direct the *single* input to *one of many* (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



$$A = S_1'S_0'Y$$

$$B = S_1'S_0Y$$

$$C = S_1S_0'Y$$

$$D = S_1S_0Y$$

Boolean expressions for the outputs

27

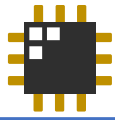
Of course, given the truth table we had earlier, we can write Boolean expressions for the output. Since we have 4 different outputs in this case, then we have 4 different expressions.

If $S_1S_0 = 00$, that implies $S_1'S_0'$, A copies whatever Y is, so the complete expression for A is $S_1'S_0'Y$.

If $S_1S_0 = 01$, that implies $S_1'S_0$, B copies whatever Y is, so the complete expression for B is $S_1'S_0Y$.

If $S_1S_0 = 10$, that implies S_1S_0' , C copies whatever Y is, so the complete expression for C is $S_1S_0'Y$.

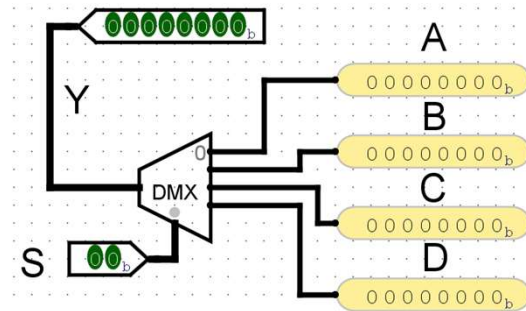
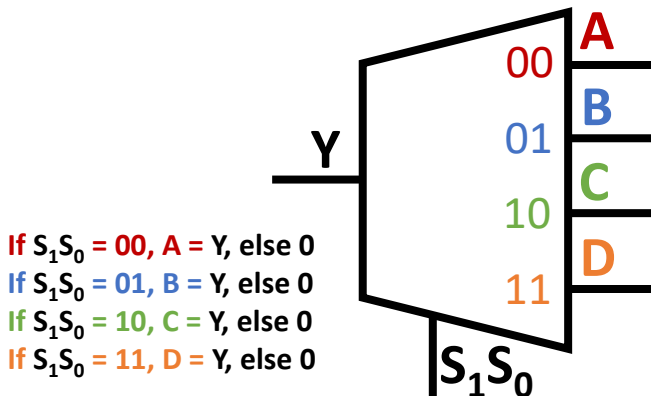
If $S_1S_0 = 11$, that implies S_1S_0 , D copies whatever Y is, so the complete expression for D is S_1S_0Y .



Demultiplexers

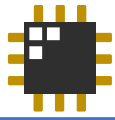
- Demultiplexers direct the *single* input to *one of many* (2^n) possible outputs using an n -bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



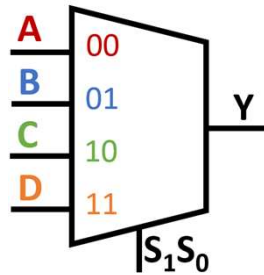
28

Here's the corresponding demonstration of a 1-to-4 demultiplexer. As you can see, the output Y is just being copied to one of the outputs depending on the value of the selector, while the other outputs remain 0 if not selected. The output corresponding to 00 is at the top, marked with 0, and then increases in binary value downward.



Multiplexers and demultiplexers

2^n -to-1 multiplexer



If $S_1S_0 = 00$, $Y = A$

If $S_1S_0 = 01$, $Y = B$

If $S_1S_0 = 10$, $Y = C$

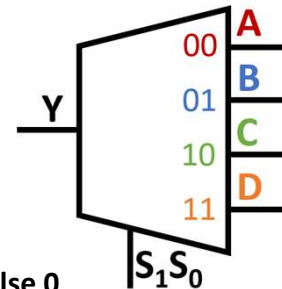
If $S_1S_0 = 11$, $Y = D$

2^n inputs

n selectors

1 output

1-to- 2^n demultiplexer



If $S_1S_0 = 00$, $A = Y$, else 0

If $S_1S_0 = 01$, $B = Y$, else 0

If $S_1S_0 = 10$, $C = Y$, else 0

If $S_1S_0 = 11$, $D = Y$, else 0

1 input

n selectors

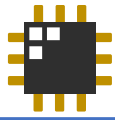
2^n outputs

29

As a summary, we have just discussed two main logic blocks so far: a multiplexer, and a demultiplexer. As you can see, they are drawn as blocks here and the internal logic gates are not shown anymore. We are already at a higher abstraction level. The two logic blocks can be differentiated by their inputs and outputs and how they behave.

A **multiplexer**, also known as a mux, is a combinational circuit that selects binary information from one of many 2^n input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of n-bit selection lines.

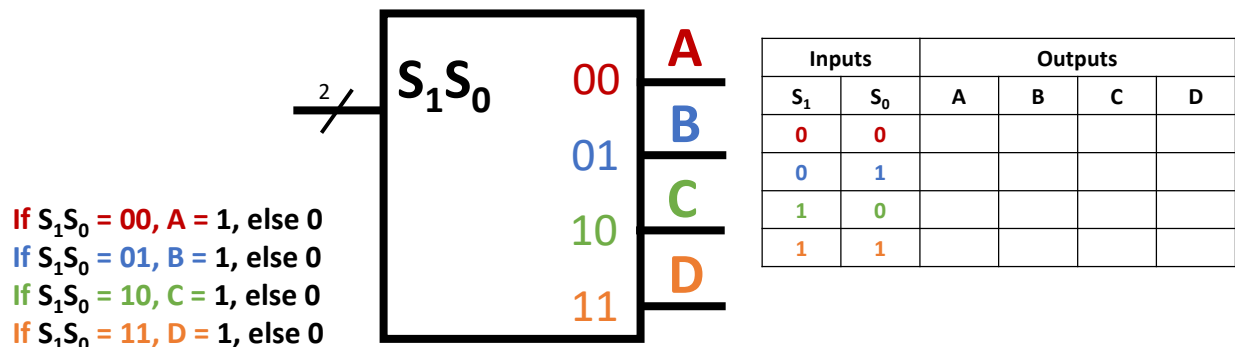
A **demultiplexer**, on the other hand, is a device that takes a single input line and routes it to one of several digital output lines. A demultiplexer of 2^n outputs has n select lines, which are used to select which output line to send the input.



Decoders

- Decoders convert/decode binary information from n input lines to 2^n unique output lines

2-to-4 decoder



30

A **decoder** is a combinational circuit that converts binary inputs into a unique output. It takes **n input lines** and activates **one of the 2^n output lines** based on the input combination.

Decoders are widely used in **address decoding, memory selection, and instruction decoding in processors.**

A 2-to-4 decoder has 2 input lines (S_1 and S_0) and 4 output lines (A, B, C, and D). The inputs S_1 and S_0 determine which output is activated. At any given time, **only one output is high (1), while the others remain low (0).**

Considering the following:

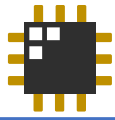
If $S_1 S_0 = 00$, $A = 1$, else 0

If $S_1 S_0 = 01$, $B = 1$, else 0

If $S_1 S_0 = 10$, $C = 1$, else 0

If $S_1 S_0 = 11$, $D = 1$, else 0

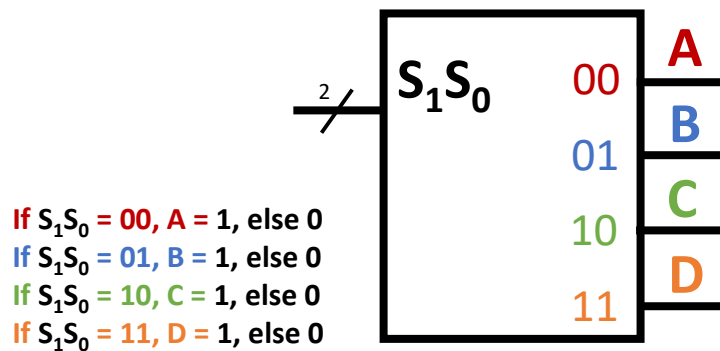
We fill up the truth table and implement the decoder in a circuit by simply following this pattern.



Decoders

- Decoders convert/decode binary information from n *input* lines to 2^n *unique output lines*

2-to-4 decoder



Inputs		Outputs			
S_1	S_0	A	B	C	D
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

31

Considering the following:

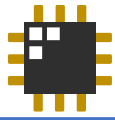
If $S_1S_0 = 00$, $A = 1$, else 0

If $S_1S_0 = 01$, $B = 1$, else 0

If $S_1S_0 = 10$, $C = 1$, else 0

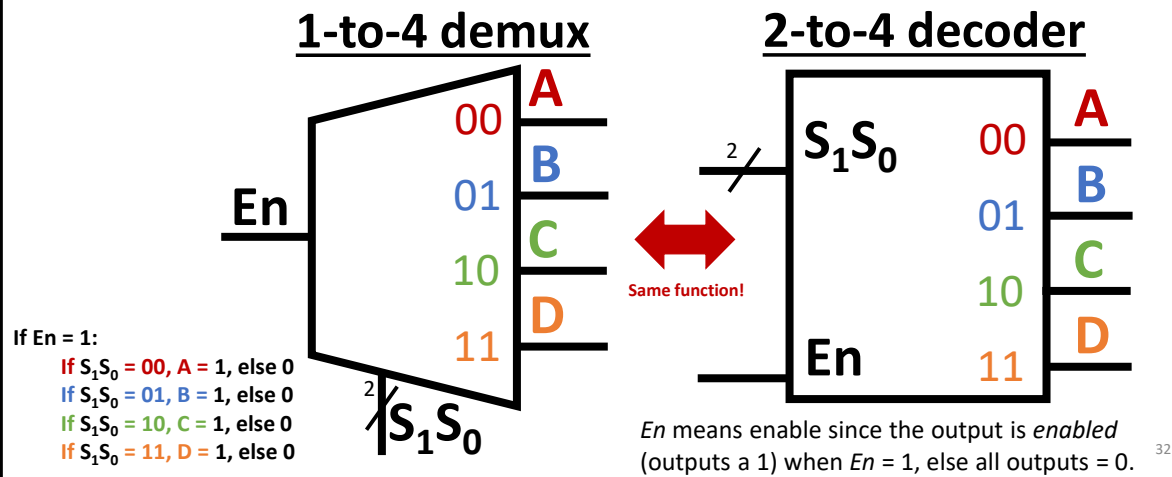
If $S_1S_0 = 11$, $D = 1$, else 0

Again, we just filled up the truth table and implement the decoder in a circuit by simply following this pattern.



Decoders and demultiplexers

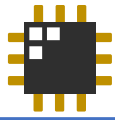
- Demultiplexers can also be thought of as *decoders with enable*
 - Drawn differently but the functionality is exactly the same!



The demultiplexers we covered earlier can also be thought of as decoders with enables. It is basically the same logic block but is drawn differently.

The enable input allows or denies the entry of data into the decoder. When the enable input is set to 1, the decoder works normally, and the binary data at the input is directed to the output line as determined by the select inputs.

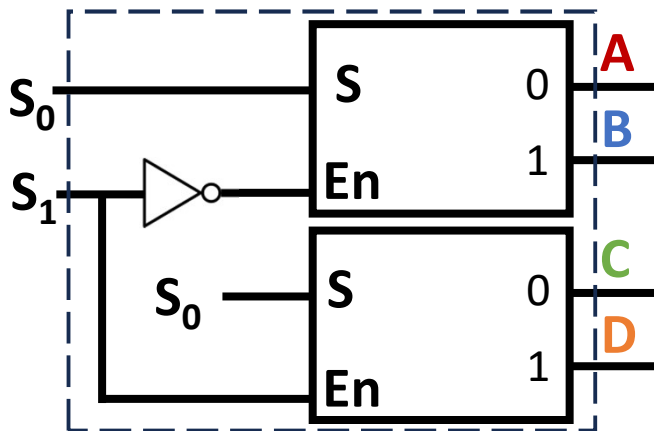
However, when the enable input is set to 0, all outputs of the decoder go to the logic 0 state, regardless of what data is coming from the input or what values the select inputs have. This is similar to the demultiplexer since the output, when selected, just copies whatever the input is, so if the input is 1 (enabled), then the selected output will copy that 1; else if the input is 0 (not enabled), then the selected output will remain 0.



Cascading decoders

- You can build larger decoders using smaller ones!

2-to-4 decoder using 1-to-2 decoders



If $S_1S_0 = 00$, $A = 1$, else 0

If $S_1S_0 = 01$, $B = 1$, else 0

If $S_1S_0 = 10$, $C = 1$, else 0

If $S_1S_0 = 11$, $D = 1$, else 0

33

Just like our exercise in building a larger multiplexer using smaller multiplexers, we can do the same of decoders. Here's a diagram of how a 2-to-4 decoder can be implemented using 1-to-2 decoders. Note how S_1 is inverted in one of the enables in the smaller decoders. This setup guarantees that only one of the smaller decoders are enabled at any given time, ensuring that all other outputs remain 0.

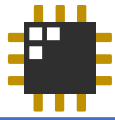
If $S_1S_0 = 00$, the smaller decoder at the top is enabled because $S_1' = 1$, then S_0 being 0 would select A to output a 1 while all the other outputs remain 0.

If $S_1S_0 = 01$, the smaller decoder at the top is enabled because $S_1' = 1$, then S_0 being 1 would select B to output a 1 while all the other outputs remain 0.

If $S_1S_0 = 10$, the smaller decoder at the bottom is enabled because $S_1 = 1$, then S_0 being 0 would select C to output a 1 while all the other outputs remain 0.

If $S_1S_0 = 11$, the smaller decoder at the bottom is enabled because $S_1' = 1$, then S_0 being 1 would select D to output a 1 while all the other outputs remain 0.

This is exactly how a 2-to-4 decoder works.

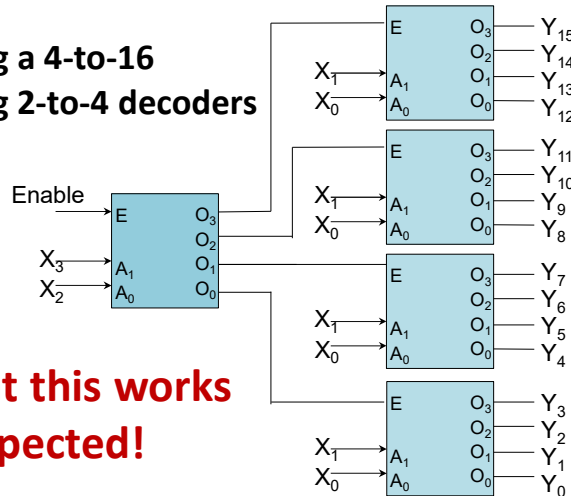


Cascading decoders

- You can build larger decoders using smaller ones!

Implementing a 4-to-16 decoder using 2-to-4 decoders

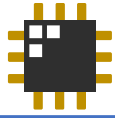
Verify that this works as expected!



X_3X_2	X_1X_0	Minterm
11	11	15
	10	14
	01	13
	00	12
10	11	11
	10	10
	01	9
	00	8
01	11	7
	10	6
	01	5
	00	4
00	11	3
	10	2
	01	1
	00	0

34

In this example, we implement a 4x16 decoder using 2x4. Following the procedure in the previous slide, we need 16 outputs while we only have 4 outputs per available decoder. Thus, we need at least $16/4 = 4$ decoders. We can check the inputs, we need $2^{(4-2)} = 2^2 = 4$ decoders. We use the MSBs (in this case the higher 4 bits) as our enable. The lowest order minterms should have $E = X_3'X_2'$. The next lower order minterms (minterms 4 to 7) will have $E = X_3'X_2$, and so on.



Truth tables using decoders

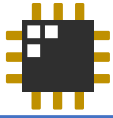
- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Implement this truth table using a decoder and a multi-input OR gate

35

Another cool thing about decoders is that implementing the corresponding logic given a truth table is actually very easy. This is because the decoder already outputs the specific per-row combinations of a truth table. This example guides you through that process.



Truth tables using decoders

- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

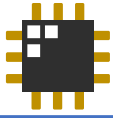
ABC
3

3-to-8 decoder		
000	$A'B'C'$	
001	$A'B'C$	
010	$A'BC'$	
011	$A'BC$	
100	$AB'C'$	
101	$AB'C$	
110	ABC'	
111	ABC	

Recall how each decoder output is already the minterm expression!

36

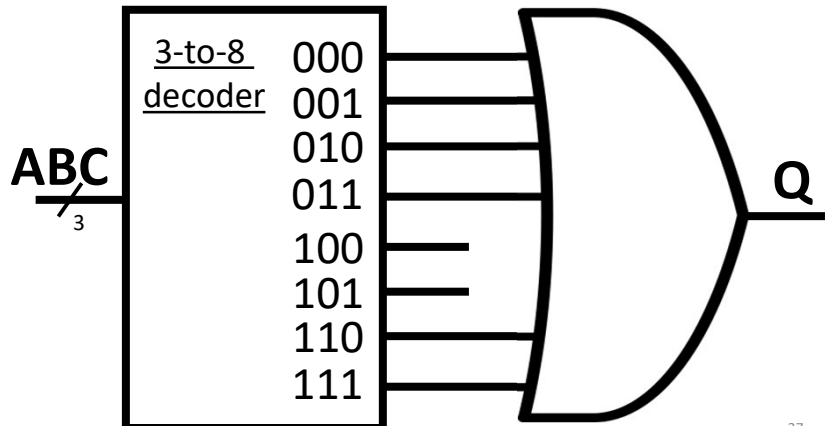
Since there are 3 inputs in the truth table, then this means we will have to use a 3-to-8 decoder. As you can see, there are 8 different outputs that we can select. Each of these outputs actually correspond to the different rows in the truth table that we are looking at right now. The only inputs that matter would be the three inputs A, B, C.



Truth tables using decoders

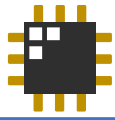
- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



37

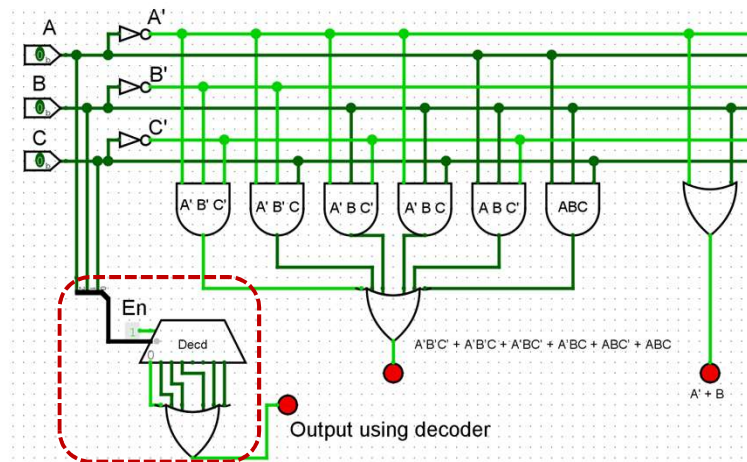
The decoder outputs become 1 when a specific condition of the inputs is satisfied. For instance, if the inputs $ABC = 000$, the first output of the decoder (at the top) is 1. That has a Boolean expression of $A'B'C'$. This is exactly how our terms would look like if we tried to analyze the truth table on its own. But now, with decoders, we can significantly speed up this process since all the input combinations are exposed as outputs. The only thing we need to do is to OR all of the relevant terms together to create the final output. The relevant terms would be those rows where the output corresponds to a 1 in the given truth table. Thus, we will end up with an implementation of a decoder and a big OR gate to combine the outputs together. Very quick and easy!



Truth tables using decoders

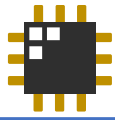
- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



38

Here's a demonstration of the decoder level implementation. This is the same circuit that we had last week. The larger one was our original expression when we just analyzed the truth table. The smaller one (with only the OR gate) is the simplified expression that we came up with after utilizing Boolean algebra to find patterns and reduce the number of terms we have in the original expression. That also took us quite a bit of time to simplify. And now, using a decoder (highlighted in a red-dashed box), we can end up with the exact same functionality and we didn't take a lot of time to implement that! That's the beauty of a decoder. It allows us to implement truth tables very quickly!



Truth tables using decoders

- Using a decoder and multi-input OR gates, implement a full adder

A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

39

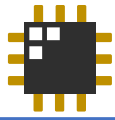
To implement a full adder using a **decoder and multi-input OR gates**, we start by analyzing the given truth table. A full adder has three inputs: **A**, **B**, and **C_{in}** (carry-in), and produces two outputs: **S** (sum) and **C_{out}** (carry-out). A **3-to-8 decoder** can be used since it takes three input lines and generates eight unique output lines, corresponding to all possible input combinations. Each output of the decoder represents one row of the truth table, activating only when its specific input combination occurs.

First, we connect **A**, **B**, and **C_{in}** as the inputs of the 3-to-8 decoder. This means each decoder output (row 0 to 7) will be active (high) for one unique input combination. Now, we need to derive expressions for **S** and **C_{out}** by identifying which rows of the truth table result in **S = 1** and **C_{out} = 1**.

From the truth table, **S = 1** when the input combinations are **(0,0,1)**, **(0,1,0)**, **(1,0,0)**, and **(1,1,1)**. To generate S, we pass these active decoder outputs through a **4-input OR gate**.

Similarly, **C_{out} = 1** when the input combinations are **(0,1,1)**, **(1,0,1)**, **(1,1,0)**, and **(1,1,1)**. To generate C_{out}, we pass these active decoder outputs through another **4-input OR gate**.

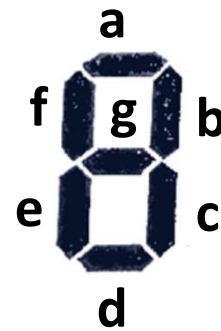
By wiring the outputs of the **3-to-8 decoder** to the inputs of two **multi-input OR gates**, we successfully implement the **sum (S) and carry-out (Cout) functions** of the full adder.



Truth tables using decoders

- Fill the table for the 7-segment decoder

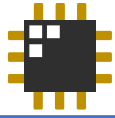
Inputs				Outputs						
W	X	Y	Z	a	b	c	d	e	f	g
0	0	0	0							
0	0	0	1							
0	0	1	0							
0	0	1	1							
0	1	0	0							
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							



Challenge: (inverted logic)
Set output to 0 if you want the LED to be on.

40

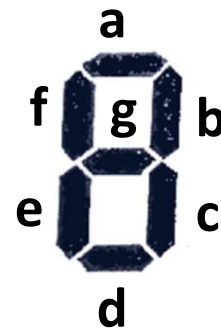
Here's another exercise. Let's say we have a seven-segment display that we want to light up with numbers 0 to 9. On the right, we are given the corresponding output assignments for each light in the seven-segment display. From this, we can start completing the table to light up the corresponding number given the 4-bit binary input W, X, Y, Z. As an additional challenge, let's use an inverted logic for this problem. This means that for a segment to light up, it must be set to 0; else, if the segment needs to be off, it must be set to 1.



Truth tables using decoders

- Fill the table for the 7-segment decoder

Inputs				Outputs						
W	X	Y	Z	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0
0	0	1	1	0	0	0	0	1	1	0
0	1	0	0	1	0	0	1	1	0	0
0	1	0	1	0	1	0	0	1	0	0
0	1	1	0	0	1	0	0	0	0	0
0	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	1	0	0

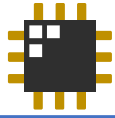


Challenge: (inverted logic)

Set output to 0 if you want the LED to be on.

41

Completing the table, we will have this. Note that we didn't exhaustively check for all the input combinations. Note how W, X, Y, Z starts with 0000 (0) and only ends with 1001 (9). Since there are four inputs, there should have been 16 different combinations all the way to 1111 (15). However, for this problem, we will assume that the inputs that will be provided is always within the range of 0-9 since that is the main purpose of a seven-segment display: to display decimal numbers.



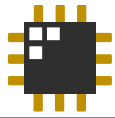
Truth tables using decoders

- Using a decoder and OR gates, implement the seven-segment decoder

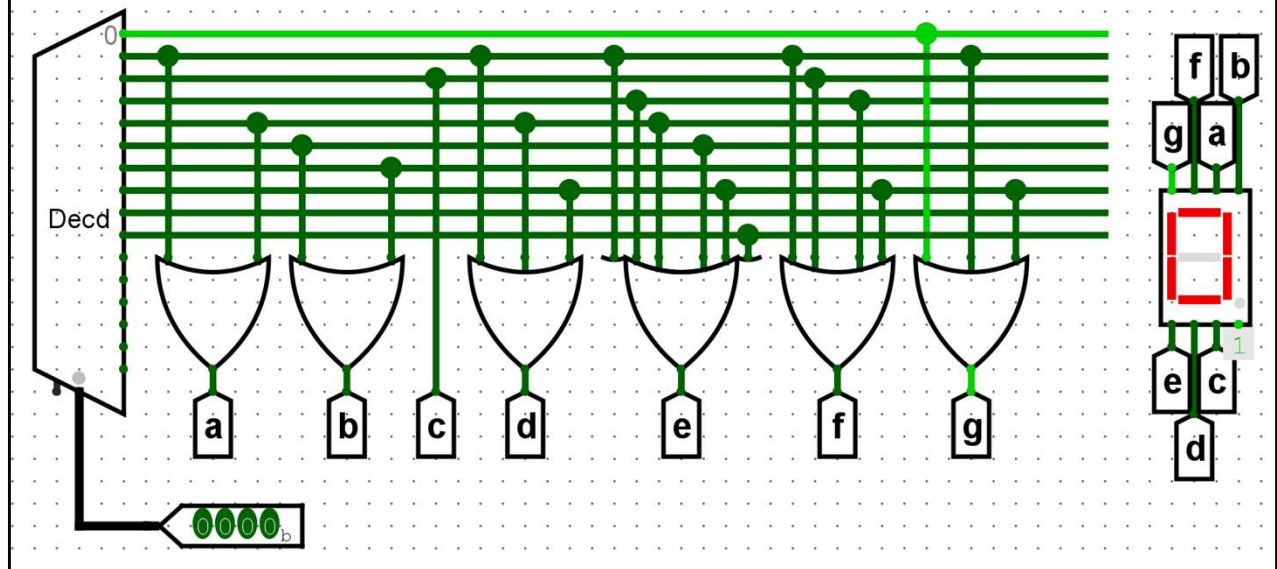
Inputs				Outputs						
W	X	Y	Z	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0
0	0	1	1	0	0	0	0	1	1	0
0	1	0	0	1	0	0	1	1	0	0
0	1	0	1	0	1	0	0	1	0	0
0	1	1	0	0	1	0	0	0	0	0
0	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	1	0	0

42

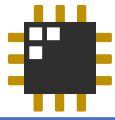
Now that we have completed the table, we can now proceed implementing the circuit using a decoder and a couple of OR gates. We have seven different outputs right now. We will have to treat them independently of each other. If you recall the process, the things that matter in this truth table is when the corresponding rows have an output of 1. For instance, the output 'a' has a 1 on input 0001 and 0100, that's it. Therefore, we will select these two outputs from our decoder and then connect them together with an OR gate. Do the same of the rest of the outputs. As you can see, the process is significantly faster since we just end up selecting which decoder outputs need to be ORed together.



Truth tables using decoders



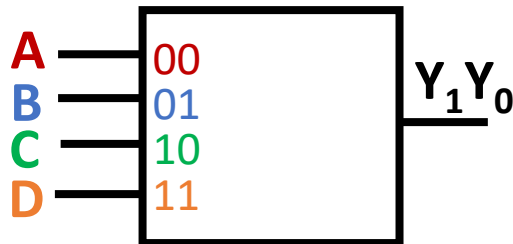
Here's a demonstration of the seven-segment display decoder, exactly following the truth table that was presented earlier. As the binary number increases from 0000 to 1001, you can see the corresponding seven-segment output on the right, consistent with the binary representation. We managed to implement this decoder very quickly with our newfound knowledge of logic blocks.



(Priority) Encoders

- Encoders are the inverse of a decoder. The n -bit output is the binary equivalent of the input (among max 2^n choices) that was asserted.

4-to-2 encoder



Inputs				Outputs	
A	B	C	D	Y ₁	Y ₀
1	X	X	X	0	0
0	1	X	X	0	1
0	0	1	X	1	0
0	0	0	1	1	1

Typically implemented as **priority encoders** since only 1 input must be followed.

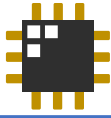
X in the truth table means don't care (it doesn't matter what the value is)

This encoder prioritizes A over the others, B over C or D, and C over D.

44

The last combinational logic block that we'll cover is an encoder. Encoders are like opposites of the decoders. They have 2^n inputs and n outputs, as shown in the slide. Note that the truth table is similar to that of the decoder, except that the outputs are now our inputs and vice versa.

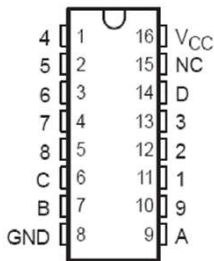
Encoders simply output the binary equivalent of the asserted input. Consequently, when two or more inputs are asserted, the output behavior of an encoder is not defined. To avoid unknown conditions when more than one input is asserted, a priority encoder may be used. In this case, the input with the highest priority is A. That is, when A is asserted, the output will be "11", regardless of the other inputs.



Priority encoders



- Encode 10-Line Decimal to 4-Line BCD
- Applications Include:
 - Keyboard Encoding
 - Range Selection



FUNCTION TABLE - '147, 'LS147

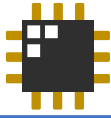
INPUTS									OUTPUTS			
1	2	3	4	5	6	7	8	9	D	C	B	A
H	H	H	H	H	H	H	H	H	H	H	H	H
X	X	X	X	X	X	X	X	L	L	H	H	L
X	X	X	X	X	X	X	L	H	L	H	H	H
X	X	X	X	X	X	L	H	H	H	L	L	L
X	X	X	X	L	H	H	H	H	H	L	L	H
X	X	X	L	H	H	H	H	H	H	L	H	L
X	X	L	H	H	H	H	H	H	H	H	L	L
X	L	H	H	H	H	H	H	H	H	H	L	H
L	H	H	H	H	H	H	H	H	H	H	H	L

H = high logic level, L = low logic level, X = irrelevant

Which input pin has the highest priority?

45

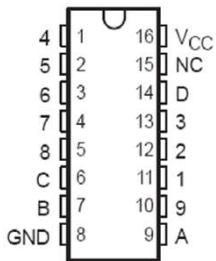
This is the truth table of the 147 encoder chip. This is an example of an active low logic, where a signal is said to be asserted or active if it is logic low (L). In this truth table, the input with the highest priority is Pin 9. Notice that when Pin 9 is asserted (L), the output is L H H L, regardless of the value of the other input pins.



Priority encoders



- Encode 10-Line Decimal to 4-Line BCD
- Applications Include:
 - Keyboard Encoding
 - Range Selection



FUNCTION TABLE - '147, 'LS147

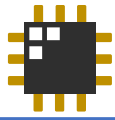
INPUTS									OUTPUTS			
1	2	3	4	5	6	7	8	9	D	C	B	A
H	H	H	H	H	H	H	H	H	H	H	H	H
X	X	X	X	X	X	X	X	L	L	H	H	L
X	X	X	X	X	X	X	L	H	L	H	H	H
X	X	X	X	X	X	L	H	H	H	L	L	L
X	X	X	X	L	H	H	H	H	H	L	H	L
X	X	X	L	H	H	H	H	H	H	L	H	L
X	X	L	H	H	H	H	H	H	H	H	L	L
X	L	H	H	H	H	H	H	H	H	H	L	H
L	H	H	H	H	H	H	H	H	H	H	H	L

H = high logic level, L = low logic level, X = irrelevant

Active-low logic! Highest priority is Pin 9

46

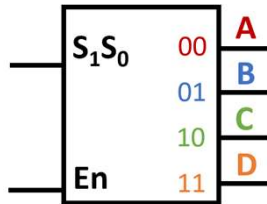
This is the truth table of the 147 encoder chip. This is an example of an active low logic, where a signal is said to be asserted or active if it is logic low (L). In this truth table, the input with the highest priority is Pin 9. Notice that when Pin 9 is asserted (L), the output is L H H L, regardless of the value of the other input pins.



Decoders and encoders

If $S_1S_0 = 00$, $A = 1$, else 0
 If $S_1S_0 = 01$, $B = 1$, else 0
 If $S_1S_0 = 10$, $C = 1$, else 0
 If $S_1S_0 = 11$, $D = 1$, else 0

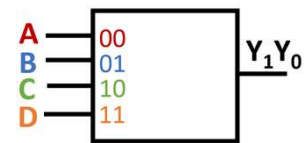
n -to- 2^n
 decoder



n inputs
 2^n output
 (1 optional enable)

Inputs				Outputs	
A	B	C	D	Y_1	Y_0
1	X	X	X	0	0
0	1	X	X	0	1
0	0	1	X	1	0
0	0	0	1	1	1

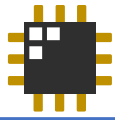
2^n -to- n
 (priority)
 encoder



2^n inputs
 n outputs

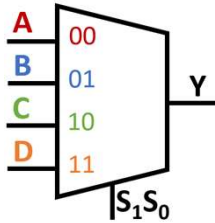
47

Again, **decoder** is a combinational circuit that takes n input lines and activates **one of 2^n output lines**. It essentially translates a **binary-coded input** into a **unique output**. Meanwhile, **encoder** performs the reverse function of a decoder: it converts multiple input lines into a **binary-coded output**. For a **priority encoder**, if multiple inputs are active, the encoder **gives priority to the highest-numbered input** and ignores the others. This ensures a **unique and deterministic output**.



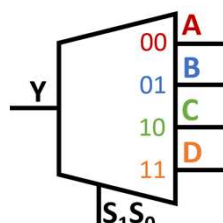
Summary

2^n -to-1
multiplexer



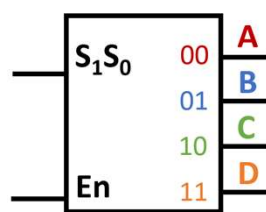
2^n inputs
n selectors
1 output

1-to- 2^n
demultiplexer



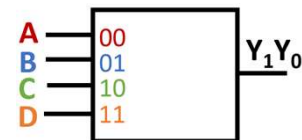
1 input
n selectors
 2^n outputs

n-to- 2^n
decoder



n inputs
 2^n output
(1 optional enable)

2^n -to-n
(priority)
encoder



2^n inputs
n outputs

48

For the final summary for this topic:

A **multiplexer (MUX)** is a circuit that selects one of multiple inputs and forwards it to a single output based on a selector signal.

A **demultiplexer (DEMUX)** performs the reverse function of a multiplexer. It takes a single input and routes it to one of multiple outputs based on select signals.

A **decoder** is another type of combinational circuit that takes binary inputs and activates one of its many outputs.

Lastly, a **priority encoder** is a special type of encoder that assigns priority to inputs. If multiple inputs are active, it encodes only the highest-priority input while ignoring the others.

These circuits play crucial roles in digital design, communication, and computing systems, ensuring efficient data handling and processing.